

# Курс: Углубленное машинное обучение и искусственный интеллект в Python

---

## Практическое задание: Полносвязные нейронные сети (FCNN) и оптимизация гиперпараметров с помощью Optuna

**Цель работы:** Научиться проектировать полносвязные нейронные сети (FCNN) для решения задачи регрессии, организовывать корректный пайплайн обработки данных и настраивать архитектуру и гиперпараметры модели с использованием библиотеки Optuna.

### Шаг 1. Поиск и выбор датасета на Hugging Face

Выберите любой открытый датасет на платформе Hugging Face, подходящий для решения задачи регрессии (с непрерывной целевой переменной).

*Примеры подходящих датасетов:*

- scikit-learn/fish-market (предсказание веса рыбы)
- cdeotte/house-prices (прогнозирование цен на недвижимость)
- mstz/california-housing (оценка стоимости жилья)
- Любой другой альтернативный датасет с числовым выходом.

**Требования к выбранному датасету:**

- Объем выборки: не менее 1000 строк.
- Количество признаков: не менее 5 числовых признаков.
- Тип задачи: строго регрессия (не классификация).

### Шаг 2. Загрузка и предварительная подготовка данных

Реализуйте загрузку данных и подготовьте их к обучению нейронной сети. Для этого выполните следующие шаги:

1. Загрузите датасет с Hugging Face и преобразуйте его в `pandas.DataFrame`.
2. Выделите целевую переменную ( $y$ ) и матрицу признаков ( $X$ ).
3. Обработайте пропущенные значения (удалите строки с пропусками или заполните

их медианными значениями по столбцам).

4. Проведите стандартизацию числовых признаков с помощью StandardScaler.
5. Разделите выборку на обучающую, валидационную и тестовую в соотношении 70% / 15% / 15%.

```
from datasets import load_dataset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Пример загрузки (замените на ваш датасет)
dataset = load_dataset("название_датасета")
df = dataset["train"].to_pandas()

# Далее выполните предобработку, масштабирование и разбиение
выборки
```

### Шаг 3. Построение базовой модели (Baseline)

Создайте и обучите базовую конфигурацию полносвязной нейронной сети со следующей архитектурой:

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(n_features,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Выходной слой для задачи регрессии
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

**Задание:** Обучите baseline-модель в течение 20 эпох. Зафиксируйте и сохраните полученное значение метрики MAE (Mean Absolute Error) на валидационной выборке.

### Шаг 4. Оптимизация гиперпараметров через Optuna

Настройте процесс автоматического подбора архитектуры и гиперпараметров сети. Вам необходимо оптимизировать следующие параметры в заданных диапазонах:

| Параметр                          | Диапазон / Значения                       |
|-----------------------------------|-------------------------------------------|
| Количество скрытых слоев          | от 2 до 5                                 |
| Количество нейронов в слое        | от 16 до 256                              |
| Скорость обучения (Learning rate) | от 1e-4 до 1e-2 (логарифмический масштаб) |
| Коэффициент Dropout               | от 0.0 до 0.5                             |
| Размер батча (Batch size)         | [16, 32, 64, 128]                         |

**Введение штрафа за сложность:** Чтобы избежать избыточного усложнения модели, измените целевую функцию оптимизации. Модель должна минимизировать не просто ошибку на валидации, а компромиссную величину:

*Целевая функция =  $val\_loss + 0.0001 * (\text{общее число параметров модели})$*

Шаблон целевой функции для Optuna:

```
import optuna

def objective(trial):
    n_layers = trial.suggest_int('n_layers', 2, 5)
    units = trial.suggest_int('units', 16, 256)
    lr = trial.suggest_float('lr', 1e-4, 1e-2, log=True)
    dropout = trial.suggest_float('dropout', 0.0, 0.5)
    batch_size = trial.suggest_categorical('batch_size', [16, 32, 64, 128])

    # Сгенерируйте модель на основе n_layers, units и dropout
    model = build_model(n_layers, units, dropout)
    model.compile(optimizer=tf.keras.optimizers.Adam(lr),
loss='mse')

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=30,
        batch_size=batch_size,
        verbose=0
    )

    val_loss = history.history['val_loss'][-1]
    total_params = model.count_params()
```

```
# Возвращаем val_loss с учетом штрафа за количество параметров
return val_loss + 0.0001 * total_params
```

Проведите серию испытаний (рекомендуется от 30 до 50 отсечек/trials) и определите лучшую комбинацию гиперпараметров.

## Шаг 5. Оценка результатов и выводы

1. Возьмите конфигурацию лучшей модели, найденную Optuna.
2. Обучите её заново на объединенной выборке (train + val).
3. Проведите финальное тестирование на отложенной тестовой выборке (test).  
Вычислите метрики MAE и R2 (коэффициент детерминации).
4. Сформулируйте краткий вывод по работе: сравните baseline-решение и оптимизированную модель. Укажите в числах и процентах, насколько применение Optuna помогло улучшить целевые метрики и как штраф за сложность повлиял на итоговую архитектуру.